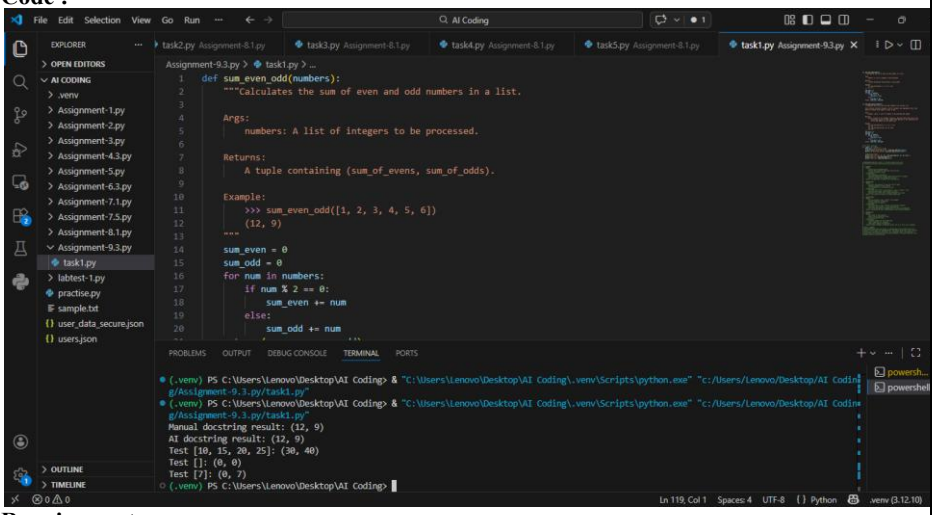
SCHOOLOFCOMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENTOFCOMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
HALLTICKET	**2303A54062**	**ASS**	9.3
Name	ANANTH	**Applicable to Batches**	47_B
AssignmentNumber:9.3(Presentassignmentnumber)/**24**(Totalnumberofassignments)			

Q.No.	Question	*ExpectedTime to complete*
	Lab 9: Documentation Generation – Automatic Documentation and Code Comments **Lab Objectives** • To understand the importance of documentation and code comments in software development • To explore how AI-assisted coding tools generate documentation and inline comments • To practice generating function-level and module-level docstrings automatically • To evaluate the quality and accuracy of AI-generated documentation • To develop a small automated documentation generator in Python	
1	**Lab Outcomes (LOs)** After completing this lab, students will be able to: • Apply AI-assisted coding tools to generate docstrings and inline comments • Analyze AI-generated documentation for correctness and readability • Create structured documentation using standard formats (Google, NumPy) • Design a mini documentation generation tool **Task 1: Basic Docstring Generation** **Scenario** You are developing a utility function that processes numerical lists and must be properly documented for future maintenance. **Prompt : Create a Python function named sum_even_odd(numbers) that takes a list of integers and returns a tuple containing the sum of even numbers and sum of odd numbers.** **1. First, write the function with a manually written Google Style docstring including:** **- Description** **- Args** **- Returns** **- Example** **2. Then generate an AI-style Google docstring for the same function separately (without changing logic).**	Week4 - Wednesday

3. Provide a structured comparison analyzing:
- **Clarity**
- **Correctness**
- **Completeness**
- **Readability**

Ensure the code runs without errors.

Code :



Requirements
• Write a Python function to return the **sum of even numbers** and **sum of odd numbers** in a given list
• Manually add a **Google Style docstring** to the function
• Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
• Compare the **AI-generated docstring** with the **manually written docstring**
• Analyze clarity, correctness, and completeness
Expected Output
• Python function with manual Google-style docstring
• AI-generated docstring for the same function
• Comparison explaining differences between manual and AI-generated documentation
• Improved understanding of AI-generated function-level documentation

Explanation : In this task, we create a function `sum_even_odd(numbers)` that finds the sum of even and odd numbers from a list and returns them as a tuple. The main focus is writing Google Style docstrings manually and then generating an AI-style docstring for the same function. Finally, both docstrings are compared based on clarity, correctness, completeness, and readability.

Task 2: Automatic Inline Comments
Scenario
You are developing a student management module that must be easy to understand for new developers.
Prompt : Create a Python class named sru_student with:

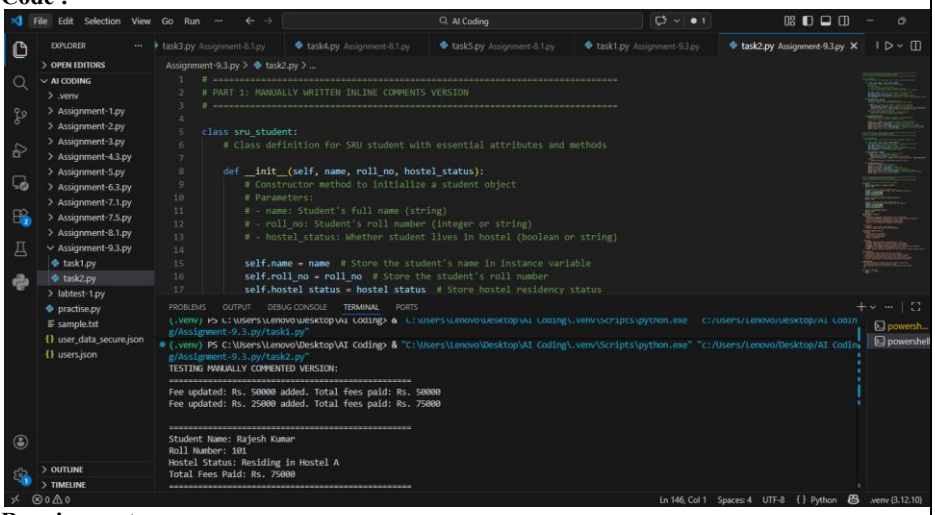
Attributes:
- **name**
- **roll_no**
- **hostel_status**

Methods:
- **fee_update(amount)**
- **display_details()**

1. First, write the class with detailed manual inline comments explaining each line or logical block.
2. Then generate an AI-assisted version of inline comments for the same code (without changing logic).
3. Provide a comparison discussing:
- Missing comments
- Redundant comments
- Incorrect explanations
- Strengths and limitations of AI-generated comments

Ensure the program runs without errors.

Code :



Requirements
• Write a Python program for an sru_student class with the following:
– Attributes: name, roll_no, hostel_status
– Methods: fee_update() and display_details()
• Manually write **inline comments** for each line or logical block
• Use an AI-assisted tool to automatically add inline comments
• Compare **manual comments** with **AI-generated comments**
• Identify missing, redundant, or incorrect AI comments
Expected Output
• Python class with manually written inline comments
• AI-generated inline comments added to the same code
• Comparative analysis of manual vs AI comments
• Critical discussion on strengths and limitations of AI-generated comments

Explanation : In this task, we create a class `sru_student` with attributes like name, roll number, and hostel status, and methods to update fees and display details. First, the code is written with detailed manual inline comments. Then AI-generated comments are written for the same code. Finally, we compare both comment styles and analyze missing, redundant, and incorrect comments.

Task 3: Module-Level and Function-Level Documentation
Scenario
You are building a small calculator module that will be shared across multiple projects and requires structured documentation.
Prompt: Create a Python calculator module containing four functions:
- add(a, b)
- subtract(a, b)
- multiply(a, b)
- divide(a, b)

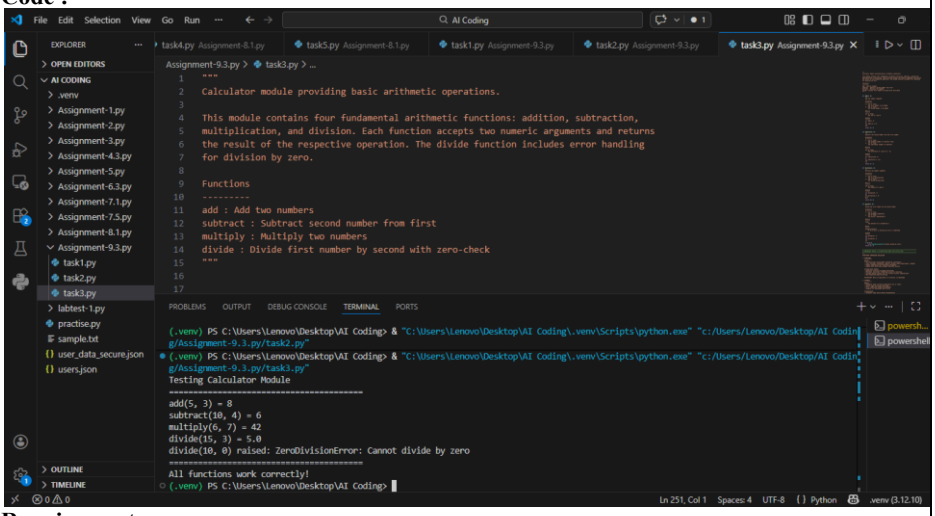
1. First, manually write NumPy Style docstrings for each function including:
- Parameters
- Returns
- Raises (for divide by zero)
- Example

2. Then generate:
- A module-level docstring
- AI-generated NumPy Style function-level docstrings (without modifying function logic)

3. Provide a structured comparison evaluating:
- Structure
- Accuracy
- Completeness
- Readability
- Professional quality

Ensure the code runs without errors.

Code :



Requirements
• Write a Python script containing **3–4 functions** (e.g., add, subtract, multiply, divide)
• Manually write **NumPy Style docstrings** for each function
• Use AI assistance to generate:
– A **module-level docstring**
– Individual **function-level docstrings**
• Compare AI-generated docstrings with manually written ones
• Evaluate documentation structure, accuracy, and readability
Expected Output
• Python script with manual NumPy-style docstrings
• AI-generated module-level and function-level documentation
• Comparison between AI-generated and manual documentation
• Clear understanding of structured documentation for multi-function scripts

Explanation : In this task, we create a calculator module with functions add, subtract, multiply, and divide. Manual NumPy Style docstrings are written for each function, including parameters, returns, raises, and examples. Then AI-generated docstrings and a module-level docstring are created. Finally, we compare manual vs AI docstrings based on

	structure, accuracy, completeness, readability, and professional quality.	
	Additional Requirement • Push the complete project documentation as a **.md file** to a GitHub repository • Ensure documentation covers module overview and function descriptions **Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots**	